

API Gateway

Learning Outcomes

After completing this lab, you will be able to:

- Register microservices with Eureka Service Registry
- Discover services using Eureka Discovery Client
- Implement an API Gateway
- Route client requests through the API Gateway
- Understand the Single Entry Point pattern

Prerequisites

Before starting this lab, you should have completed Lab 1, where you created:

- Employee Service
- Customer Service
- Product Service
- Order Service

Each microservice should:

- expose REST APIs
- use MongoDB Atlas
- have its own database
- run successfully on separate ports

Required Software

- IntelliJ IDEA
- Java 17
- Maven
- MongoDB Atlas
- Postman

Existing Project Structure

Use the project created in Lab 1. Your project should now look like:

Microservices-Lab

- employee-service
- customer-service
- product-service
- order-service

Part 1: Create Service Registry

Create a new Spring Boot project using spring-boot [initializr](#) using Maven, Java 17, Spring Boot 4.0.6/4.1.0.

Name: service-registry

Dependencies: Eureka Server

Place this project inside the same parent folder.

Add an application.yml file under resources and add:

```
server:
  port: 8761

spring:
  application:
    name: SERVICE-REGISTRY

eureka:
  client:
    register-with-eureka: false
    fetch-registry: false
```

ServiceRegistryApplication.java:

```
@SpringBootApplication
@EnableEurekaServer
public class ServiceRegistryApplication {

    public static void main(String[] args) {
        SpringApplication.run(ServiceRegistryApplication.class, args);
    }

}
```

Now run ServiceRegistryApplication and open <http://localhost:8761> on your browser. The Eureka dashboard should appear.

Part 2: Register Existing Microservices

Open the Employee Service created in Lab 1.

Open **application.yml** and change only these two lines:

```
register-with-eureka: true
fetch-registry: true
```

These were set to false previously. Now we set them to true in order to register with the Service Registry. Everything else should stay the same.

Do the same for all the other microservices.

Run the services one by one. Verify that all services appear in Eureka with the status 'UP'.

Part 2: Create API Gateway

Create a new Spring Boot project using spring-boot [initializr](#) using Maven, Java 17, Spring Boot 4.0.6/4.1.0.

Name: api-gateway

Dependencies: Reactive Gateway, Eureka Discovery Client

Place this project inside the parent folder.

Add an application.yml file under resources and add:

```
server:
  port: 8080

spring:
  application:
    name: API-GATEWAY

cloud:
  gateway:
    server:
      webflux:
        routes:
          - id: employee-service
            uri: lb://EMPLOYEE-SERVICE
            predicates:
              - Path=/employees/**

          - id: customer-service
            uri: lb://CUSTOMER-SERVICE
            predicates:
              - Path=/customers/**

          - id: product-service
            uri: lb://PRODUCT-SERVICE
            predicates:
              - Path=/products/**

eureka:
  client:
    register-with-eureka: true
    fetch-registry: true
    service-url:
```

```
defaultZone: http://localhost:8761/eureka/
```

ApiGatewayApplication.java:

```
@SpringBootApplication
public class ApiGatewayApplication {

    public static void main(String[] args) {
        SpringApplication.run(ApiGatewayApplication.class, args);
    }

}
```

Now run ApiGatewayApplication. It should appear alongside all the other microservices on your Eureka Dashboard.

Part 3: Verify API Gateway

Open Postman and instead of calling

http://localhost:9001/employees/1

try calling:

http://localhost:8080/employees/1

The API Gateway should forward the request to Employee Service. Check whether all of your requests are working when sent via the API Gateway.